Core 90 - Sorting and divide and conquer

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Sorting and divide and conquer | 2 questions | tap a title to jump to it

M 1. Find Containing Range — **PracHub 2026-06-04**

M 2. Find shared objects across two log files — **PracHub 2025-11-05**

STUDY PATH: question 84 of 167 (Core 90)

Priority score 4.61 | Sorting and divide and conquer | asked 138 days ago

MEDIUM | PracHub | Technical Screen | 1 of 2

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 65.2** | **30 DAYS 41.1** | **PRIORITY 12/228**

ABOUT THIS QUESTION

Topic: Core 90 - Sorting and divide and conquer

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

APPROACH

Sort/merge ranges once, then answer each query with binary search on starts and check the end. O(n log n) preprocessing, O(log n) per query.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the given ranges can overlap, what to return when a point lies in several ranges (the original range, the merged range, or all of them), and what to return when none contains it.

BRUTE FORCE TO SAY OUT LOUD

For each query scan all ranges. O(n) per query.

OPTIMAL IDEA

Sort the ranges by start and merge overlaps once, O(n log n). For each query binary search for the last merged range whose start is at most the point, then check that the point is at most its end. O(log n) per query. If the original (unmerged) range is required and ranges may overlap, use an interval tree or keep the maximum end seen in a prefix array.

DRY-RUN THESE CASES

A point before all ranges, after all ranges, in a gap, exactly on an endpoint, inside two overlapping ranges.

LIKELY FOLLOW-UPS

Insert and delete ranges (a balanced tree), or many point queries at once (sort and sweep).

PITFALLS

INTERVIEW-DAY PLAN

Binary searching without merging when ranges overlap. Mishandling the endpoint rule.

HOW TO THINK ABOUT IT

Merge the ranges once so they are disjoint and sorted; then 'which range contains x' is 'the last range whose start is at most x, and does its end reach x'. That is one binary search per query. Be clear whether the answer is the original range or the merged one, since they differ when ranges overlap.

Basis: Written (compare LeetCode 56 and a binary search for the range). Assumed rule (confirm it): given possibly overlapping ranges, answer queries 'which merged range contains x?', or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def find_containing(ranges, x):
    for start, end in ranges:
        if start <= x <= end:
            return (start, end)
    return None
```

TIME	SPACE
O(n) per query	**O(n)**

WHY IT IS SLOW

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION · Merge once, then binary search the starts

TIME	SPACE
O(n log n) to build, O(log n) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

FOLLOW-UP 2 Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

FOLLOW-UP 3 Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Containing Range
#
# This question evaluates the ability to design
# appropriate data structures and apply algorithmic
# reasoning for efficient interval containment queries,
# emphasizing preprocessing, correctness, and time-space
# trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return every original range that contains x, not the
# merged one.
# 2. Ranges are inserted and deleted over time.
# 3. Queries are given in bulk.
#
# INTERVIEW NOTES
# Merge the ranges once so they are disjoint and sorted;
# then 'which range contains x' is 'the last range whose
# start is at most x, and does its end reach x'. That is
# one binary search per query. Be clear whether the answer
# is the original range or the merged one, since they
# differ when ranges overlap.
#
# BRUTE FORCE (say it first): Scan all ranges for each
# query
# Merge the ranges (or not) and for each query test every
# range for containment. Correct and the oracle; each
# query walks the whole list. Time O(n) per query. Space
# O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): Scan all ranges for each
# query | time O(n) per query
def containing_range_scan(ranges, x):
    merged = []
    for s, e in sorted(ranges):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    for s, e in merged:
        if s <= x <= e:
            return (s, e)
    return None


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
# FOLLOW-UP 1: Return every original range that contains
# x, not the merged one.
# Answer: Sort by start and consider only ranges starting
# at or before x (one bisect); among those, keep the ones
# whose end is at least x. With many overlaps use an
# interval tree for O(log n + k) per query.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    # FOLLOW-UP 2: ranges can be inserted over time
    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None

    # FOLLOW-UP 1: every ORIGINAL range containing x
    def find_original(self, x):
        # every ORIGINAL range that contains x
        k = bisect_right([s for s, _ in self.original], x)
        return [r for r in self.original[:k] if r[1] >= x]

    # FOLLOW-UP 3: queries given in bulk
    def find_many(self, queries):
        # bulk: sort the queries and sweep once
        order = sorted(range(len(queries)),
                       key=lambda i: queries[i])
        out, j = [None] * len(queries), 0
        for i in order:
            x = queries[i]
            # skip ranges that end before x
            while j < len(self.ends) and self.ends[j] < x:
                j += 1
            inside = j < len(self.starts)
            if inside and self.starts[j] <= x:
                out[i] = (self.starts[j], self.ends[j])
        return out


if __name__ == "__main__":
    idx = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert idx.find(4) == (1, 6)
    assert idx.find(7) is None
    assert idx.find(8) == (8, 10)
    assert idx.find(0) is None and idx.find(99) is None
    # the original ranges that cover 2
    assert idx.find_original(2) == [(1, 3), (2, 6)]
    # touching ranges merge: (6, 8) joins the first two
    # blocks
    idx.insert(6, 8)
    assert idx.find(7) == (1, 10)
    assert idx.find(12) is None
    qs = [4, 20, 16, 9]
    assert idx.find_many(qs) == [idx.find(q) for q in qs]
```

FUNCTION AND TEST CASES

Signature

```
def solution(ranges: list[list[int]], queries: list[int])
-> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

STUDY PATH: question 85 of 167 (Core 90)

Priority score 3.52 | Sorting and divide and conquer | asked 349 days ago

MEDIUM | PracHub | Technical Screen | 2 of 2

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE	ALL TIME 50.4	30 DAYS 46.9	PRIORITY 23/228

ABOUT THIS QUESTION

Topic: Core 90 - Sorting and divide and conquer

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Posted on PracHub: 2025-11-05 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

QUESTION

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']],
    [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

APPROACH

Load the smaller file's obj_ids into a hash set, stream the other and collect matches; if both are huge use external sort/merge or hash partitioning to disk (or a Bloom filter prefilter).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to report distinct ids or every occurrence, how large each file is, whether order matters, and what to output (ids only or full lines).

BRUTE FORCE TO SAY OUT LOUD

For each id in file A scan file B. O(n * m), unusable at scale.

OPTIMAL IDEA

Load the ids of the smaller file into a hash set, stream the larger file and collect hits (O(n + m) time). If both do not fit in memory, hash-partition both files by id into buckets on disk so each bucket pair fits, then join bucket by bucket; or external sort both and merge. A Bloom filter of one file prefilters the other.

DRY-RUN THESE CASES

Empty file, duplicate ids, identical files, no overlap, ids that differ only in case or whitespace.

LIKELY FOLLOW-UPS

Distributed version (shuffle by key), top N shared ids by frequency, restrict to a time window.

PITFALLS

Loading both files into memory. Not deduplicating output.

HOW TO THINK ABOUT IT

A common key in two files is a join. If one file fits in memory, load its keys into a set and stream the other. If neither fits, split both files by hash of the key into buckets small enough for memory; equal keys always land in the same bucket number, so you only compare bucket i of one file with bucket i of the other (Grace hash join).

Basis: Written; a join of two large relations (external hash join). Line format assumed: 'timestamp obj_id other'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(file_a, file_b):
    shared = set()
    for a in file_a:
        for b in file_b:
            if a.obj_id == b.obj_id:
                shared.add(a.obj_id)
    return shared
```

TIME	SPACE
O(n * m)	O(1) extra

WHY IT IS SLOW

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION · Hash set of the smaller file's ids

TIME	SPACE
O(n + m)	O(distinct ids of the smaller file)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

FOLLOW-UP 2 Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

FOLLOW-UP 3 A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. **Cost:** O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shared objects across two log files
#
# You are given two large log files representing activity
# on two different days. Each line of each log has three
# fields: - timestamp: a time value (you may treat it as
# an opaque string or integer) - obj_id: identifier of an
# object (string or integer) - client_id: identifier of a
# client (string or integer) The logs are not necessarily
# sorted by any field. Define an object as **interesting**
# if: 1. Its obj_id appears at least once in day 1's log
# **and** at least once in day 2's log; and 2. Across all
# of its appearances in both days combined, it is
# associated with at least **two distinct** client_id
# values. ### Tasks 1. Design and implement a function
# that, given file handles (or streaming iterators) for
# the two log files, returns the set of all such
# interesting obj_ids. 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both files are too big for memory.
# 2. Want objects seen on both days within an hour of each
# other.
# 3. A tiny memory budget and an approximate answer is
# fine.
#
# INTERVIEW NOTES
# A common key in two files is a join. If one file fits in
# memory, load its keys into a set and stream the other.
# If neither fits, split both files by hash of the key
# into buckets small enough for memory; equal keys always
# land in the same bucket number, so you only compare
# bucket i of one file with bucket i of the other (Grace
# hash join).
#
# BRUTE FORCE (say it first): Compare every pair of lines
# For each line in A scan all of B for the same object id.
# Correct, but quadratic in the file sizes. Time O(n * m).
# Space O(1) extra.


import zlib
from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair of lines
```

```
# | time O(n * m)
def shared_brute(a_lines, b_lines):
    out = set()
    for x in a_lines:
        for y in b_lines:
            if x.split()[1] == y.split()[1]:
                out.add(x.split()[1])
    return out


# HELPERS for follow-up 1
def bucket_of(obj, buckets):
    # a stable hash (Python's own hash changes between
    # runs)
    return zlib.crc32(str(obj).encode()) % buckets


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
```

```python
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)


# OTHER WAY: external sort of both days
def shared_objects_sorted(day1, day2):
    # the other way: external sort both days by object,
    # then a
    # merge that holds only one object's clients at a time
    a = sorted(day1, key=lambda r: r[1])
    b = sorted(day2, key=lambda r: r[1])
    i = j = 0
```

```python
    out = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj, clients = a[i][1], set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                out.append(obj)
    return out


# FOLLOW-UP 1: Both files are too big for memory.
# Answer: Hash-partition both files into B buckets on disk
# by hash of the object id, then process bucket pairs one
# at a time with the in-memory set join. Choose B so that
# one bucket of the smaller file fits in memory.
def shared_objects_partitioned(day1, day2, buckets=4):
    # memory limit: write each row to a bucket "file"
    # chosen by
    # the hash of its object, then solve one bucket at a
    # time.
    # All rows of an object land in the same bucket.
    parts1 = [[] for _ in range(buckets)]
    parts2 = [[] for _ in range(buckets)]
    for rows, parts in ((day1, parts1), (day2, parts2)):
        for row in rows:
            parts[bucket_of(row[1], buckets)].append(row)
    out = []
    for b in range(buckets):
        out += shared_objects(parts1[b], parts2[b])
    return sorted(out)


# FOLLOW-UP 3: A tiny memory budget and an approximate
# answer is fine.
# Answer: Build a Bloom filter from one file and test the
# other against it, accepting a small false-positive rate;
```

```python
# verify the few hits exactly in a second pass if needed.
class Bloom:
    """Tiny memory, approximate: may say 'yes' for a few
    objects that were never added."""

    def __init__(self, bits=1 << 16, hashes=3):
        self.bits, self.hashes = bytearray(bits), hashes

    def spots(self, key):
        for seed in range(self.hashes):
            text = f"{seed}:{key}".encode()
            yield zlib.crc32(text) % len(self.bits)

    def add(self, key):
        for s in self.spots(key):
            self.bits[s] = 1

    def maybe(self, key):
        return all(self.bits[s] for s in self.spots(key))


if __name__ == "__main__":
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    d2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]
    assert shared_objects(d1, d2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []
    assert shared_objects_partitioned(d1, d2, 3) == ["o1"]
    assert shared_objects_sorted(d1, d2) == ["o1"]
    many1 = [[i, f"o{i % 7}", f"c{i % 3}"]
             for i in range(40)]
    many2 = [[i, f"o{i % 5}", f"c{i % 2}"]
             for i in range(40)]
    want = shared_objects(many1, many2)
    parts = shared_objects_partitioned(many1, many2, 5)
    assert parts == want
    assert shared_objects_sorted(many1, many2) == want
    bloom = Bloom()
    for row in d1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and not bloom.maybe("o999")
```

ALSO REPORTED · 1 more

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.

Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).

The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).

For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

ALSO REPORTED · 1 more

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.